

communication between the laptop GUI and the Arduino board. The GUI sends control commands to the Arduino via the USB serial port. Upon receiving these commands, the Arduino interprets them and activates the motor using the L293D driver. Specifically:

- Command 0: Moves the motor forward
- Command 1: Stops the motor
- Command 2: Reverses the motor direction
- Commands 3 through 255: Represent PWM values for speed control

Software

Two code sets are used in this system:

1. Arduino motor command control code
2. HMI code for the laptop

Arduino motor command control code. The Arduino motor command control code enables the Arduino Uno to receive and interpret serial commands from the laptop-based GUI and control the motor accordingly. To operate, the Arduino .ino file must first be downloaded. Once opened in the Arduino IDE, the

correct board and COM port should be selected. After configuration, the code can be compiled and uploaded to the Arduino Uno. A snippet of this code is shown in Fig. 3.

HMI code. The second part of the system involves the GUI, developed using Processing. This interface includes buttons and sliders that send user-defined commands to the Arduino. To get started, Processing must be downloaded and installed from <https://processing.org/download/>. Processing is a flexible software sketchbook and a program-

```
1 if((forward && !_stop && !reverse) && ( val > 2 && val <= 255))
2 {
3   digitalWrite(IN1, LOW);
4   analogWrite(IN2, val);
5 }
6 if(!_forward && !_stop && !reverse)
7 {
8   digitalWrite(IN1, LOW);
9   digitalWrite(IN2, LOW);
10 }
11 if((!forward && !_stop && reverse) && ( val > 2 && val <= 255))
12 {
13   analogWrite(IN1, val);
14   digitalWrite(IN2, LOW);
15 }
```

Fig. 3: Snippet of Arduino motor command control code

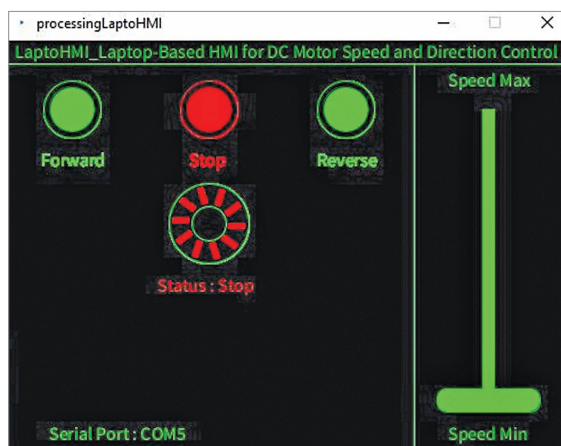


Fig. 5: Laptop HMI panel

```
void draw()
{
  if ((mousePressed == true) && (mouseX>40 && mouseX<80) && (mouseY>40 && mouseY<80))
  {
    forwardButtonPressed();
    myPort.write(0); // Command 0 for selection of Forward mode.
    control = 315; // Resetting the Control bar
    speed = 3; // Resetting the speed
  } else if (mousePressed==false) forwardButton();

  if ((mousePressed == true) && (mouseX>160 && mouseX<200) && (mouseY>40 && mouseY<80))
  {
    stopButtonPressed();
    myPort.write(1); // Command 1 for selection of Stop.
    control = 315; // Resetting the Control bar
    speed = 3; // Resetting the speed
  } else if (mousePressed==false) stopButton();

  if ((mousePressed == true) && (mouseX>280 && mouseX<320) && (mouseY>40 && mouseY<80))
  {
    reverseButtonPressed();
    myPort.write(2); //Command 2 for selection of reverse mode.
    control = 315; // Resetting the Control bar
    speed = 3; // Resetting the speed
  } else if (mousePressed==false) reverseButton();
  speedControl();
}
```

Fig. 4: Snippet of the Processing code

ming language designed for learning and creating visual arts-based applications. In this system, it is used to send serial commands to the Arduino for controlling motor direction (forward, reverse, stop) and speed (via PWM). A snippet of the Processing code is shown in Fig. 4.

The GUI provides interactive buttons and sliders to send direction and speed commands to the Arduino. It communicates via the serial port, enabling real-time control. This removes the need for manual switches or physical controllers. Fig. 5 shows the laptop HMI panel.

Construction of HMI. In Fig. 5, the interface includes